159.102 NOTES · WEEK 8 EXAM REVISION

Recursion, Searching & Backtracking

A quick-glance recap of Section 8: how recursive functions call themselves and stop at a base case, how linear and binary search find an item in an array, and how backtracking finds a path through the caves.

159.102 Applied Programming

Compiled from the Section 8 lecture notes

October 2026

1. Recursion - The Core Idea

Recursion happens when something is defined in terms of itself. A **recursive function** is simply a function that **calls itself**. Recursion is an alternative to loops: the same problem can usually be solved both ways, and sometimes the recursive version is shorter and more natural than the loop version.

Every recursive function must have a **base case** - the one situation where the function does **not** call itself. The base case is the stop signal: it is what makes the recursion end. Without it, the function would keep calling itself forever and the program would crash.

The classic example is **factorial**. Factorial multiplies a number by every whole number below it, and it hides a smaller factorial inside itself: 5! = 5 × 4 × 3 × 2 × 1 = 120, and because 4 × 3 × 2 × 1 is just 4!, we get the recursive definition **x! = x × (x-1)!** with base case **0! = 1**.

```
int factorial(int x) {
    if (x == 0) {
        return 1;                    // the base case - stops recursion
    }
    return x * factorial(x - 1);     // the function calls itself
}
```

Key points to remember:

- The function calls itself - that is the classic definition of recursion.
- There are no loops in this program; recursion replaces them.
- The base case is an if-statement with a return, placed before the recursive call.
- Factorials grow very fast: this program only handles n up to 16 (try 17 and the int overflows).
- We do not use recursion for speed - it wastes time on many function calls. We use it when it makes the solution simpler.

2. How a Recursive Call Runs

To calculate factorial(3), the computer makes a chain of calls. Each call cannot finish until the next one answers, so every call **pauses and remembers where it is**. The chain goes down until the base case answers, then the answers travel back up:

Step	Call	What happens
down 1	factorial(3)	needs 3 × factorial(2) → pauses and waits
down 2	factorial(2)	needs 2 × factorial(1) → pauses and waits
down 3	factorial(1)	needs 1 × factorial(0) → pauses and waits
down 4	factorial(0)	base case! returns 1
up 3	factorial(1)	1 × 1 = 1 → returns 1
up 2	factorial(2)	2 × 1 = 2 → returns 2

Step	Call	What happens
up 1	factorial(3)	3 × 2 = 6 → returns 6 to main

factorial(3) traced step by step

All those paused calls are saved in a part of memory called the **stack**. Each paused call keeps its own local variables and parameters there, like a pile of plates - one plate per call. If a program makes too many recursive calls, the pile grows too tall and a **stack overflow** error occurs. This is why the base case matters so much: it is the thing that guarantees the pile will start shrinking again.

3. Recursion vs Loops

There are two broad approaches to programming. An **iterative** algorithm uses loops (for, while) to repeat instructions. A **recursive** algorithm uses functions that call themselves instead. Most programs written before this section were iterative. Both roads usually lead to the same answer - the skill is knowing both exist and picking the easier one for the problem at hand.

```
// Iterative countdown: repeat one action with a loop
for (int i = n; i >= 1; i--) cout << i << endl;

// Recursive countdown: print one number, hand the rest to a copy of me
void countdown(int n) {
    if (n == 0) return;            // base case
    cout << n << endl;
    countdown(n - 1);              // smaller problem
}
```

	Iterative (loops)	Recursive (self-calls)
Speed	Faster - no call overhead	Slower - many function calls
Style	Sometimes long and messy	Often short and natural
Factorial	Easy with a for loop	Two lines with a base case
Backtracking	Extremely difficult	The natural fit

Choosing between the two approaches

> **Why bother with recursion?** Not speed - recursion is slower. We use it when the alternative algorithm is very complicated. Backtracking (the caves problem below) is the example where recursion is natural and writing it with loops would be extremely difficult.

4. Linear Search

Searching is one of the most common things computers do - looking up student records, bank accounts, medical records. **Linear search** is the simplest method: start at the first element (index 0), move through the array in a line, and check each element against the target. "Linear" just means "in a line".

Example array: **846 109 528 761 391 220**. Searching for 761 checks 846 (no), 109 (no), 528 (no), then 761 - found at **index 3**. Searches return an **index**, because from the index you can reach the whole record. If the target is not in the array, the function returns **-1**, which can never be a real index.

```
int linearsearch(int first, int last, int target) {
    indx = -1;                          // assume not found
    for (i = first; i <= last; i++) { // walk the line, first to last
        if (target == id[i]) {
            indx = i;                   // found it - remember the index
        }
    }
    return indx;                        // index, or -1
}
```

- Biggest advantage: works on random data - **no sorting needed**.
- Very easy to program, so fewer chances of mistakes.
- Slow on big arrays - in the worst case it checks every single element.
- The parameters first and last are indexes; they say which part of the array to search.

5. Binary Search

For a much larger array, **binary search** is far faster. It works like the "guess my number" game or a phone book: look in the **middle**, and the comparison tells you which half to throw away. Each check removes half of the remaining data, so 1000 items take only about 10 checks.

The one big rule: the data must be sorted. Sorting is what makes it safe to discard half the array. Worked example - searching for 761 in the sorted array **109 220 391 528 761 846**: the middle of 0-5 is index 2 (value 391); 761 is bigger, so search the right half. The middle of 3-5 is index 4 (value 761) - match! Found at index 4 in **2 checks** (linear search needed 5).

```
int binarysearch(int first, int last, int target) {
    if (first > last) {
        return -1;                  // base case: nothing left to search
    }
    middle = (first + last) / 2;    // look in the middle
    if (target == id[middle]) {
        return middle;              // found it
    } else if (target > id[middle]) {
        return binarysearch(middle + 1, last, target);   // right half
    } else {
        return binarysearch(first, middle - 1, target);  // left half
    }
}
```

	Linear Search	Binary Search
Speed	Slow - may check every item	Fast - halves the field each step
Data	Any order, no sorting needed	Must be sorted
Code	Very easy to write	Trickier - recursive, no loops
Use when	Data changes often, small arrays	Large, stable, sorted data

Linear vs binary search at a glance

> **Favourite exam trap:** "Must the data be sorted?" Linear search - **no**. Binary search - **yes**. Also remember the advice from the notes: experienced programmers rate "easy to program" highly, but if speed matters on big sorted data, binary search wins every time.

6. Backtracking - The Caves Problem

Backtracking means: attempt a path, realise the current attempt will fail, go back to a previous situation, and try a different option. It is used in many games and Artificial Intelligence problems. The notes model it with a system of caves joined by **one-way doors**, and ask: is there a path from cave A to cave B?

The recursive function findpath(here, last) reads in plain English: "Am I at the goal? If yes - success (base case). Otherwise, for each door in this cave, send a copy of myself through it. If a copy succeeds, pass success back up. If every door fails, this branch is a dead end." When a call fails, the caller simply moves on to its **next door** - and that automatic rewinding **is** the backtracking.

The problem - walking in circles

Doors can loop: cave 0 → 4 → 3 → 0 → 4 → 3 ... forever. Version 1 of the program can get stuck in this cycle and never find a path that exists (for example a path from cave 0 to cave 1).

The fix - carry chalk

Mark every cave you enter. If you arrive at a cave that is already marked, skip it - you have been here and it led nowhere new. Before starting a new search, clear all the marks. In code: add a boolean marked item to each cave, check it at the top of findpath, set it before checking doors, and wipe all marks at the start.

```
bool marked[500];                   // one chalk mark per cave

bool findpath(int here, int last) {
    if (here == last)  return true;   // base case: at the goal
    if (marked[here])  return false;  // been here already - skip
    marked[here] = true;              // leave a chalk mark
    for (int i = 0; i < numdoors[here]; i++) {
        if (findpath(doors[here][i], last)) {
            return true;              // a door worked - pass success up
        }
    }
    return false;                     // dead end: all doors failed
}
```

Trace with chalk, from cave 0 to cave 1: enter 0, mark it, try its first door. Enter 4, mark it, try its door. Enter 3, mark it, try door to 0 - already marked, skip. Cave 3 backtracks and tries its **next** door. Enter 2, mark it, try its door. Enter 1 - that is the goal! True travels all the way back up: path found (0 → 4 → 3 → 2 → 1). The mark plus the backtracking work as a team.

Real-life versions of the same problem: a GPS finding a route between two buildings, a milk company planning the most efficient route between towns, laying the cheapest fibre-optic cable between buildings, and university timetabling - when class times clash, backtrack, remove some classes and try different times.

7. Exam Quick-Fire Facts

One line each - read this table in the minutes before you walk into the exam room:

Fact	Memory hook
Recursion	A function that calls itself
Base case	The stop signal - the one case that does not call itself; write it before the recursive call
Stack	Memory that saves each paused call; too many calls = stack overflow
Iterative vs recursive	Loops vs self-calls - two roads to the same answer
Search return value	An index; -1 means "not found"
Linear search	Checks items in a line; works on unsorted data
Binary search	Checks the middle, throws away half; data MUST be sorted
Backtracking	Try a path, fail, go back, try the next door
Marking caves	Chalk on the wall - skip visited caves; stops infinite loops
Why recursion?	Not speed - simplicity, when it fits the problem naturally

Ten facts worth memorising

> **Three final tips:** 1) Find the base case first - every recursive function on the exam has one. 2) "Must the data be sorted?" Linear: no. Binary: yes. 3) Clear all cave marks before starting a new search, or it will think every cave has already been visited.